

How to fine-tune Phi-3 Vision on a custom dataset

Here's how to fine tune a state of the art multimodal LLM on a custom dataset

[Brett Young](#)

 Share

 4 comments

 8 stars



Created on May 28 | Last edited on September 27

In this blog post, we'll be **fine-tuning Phi-3 Vision**, a model is capable of synthesizing text from image data.

The main goal is to create a system that can generate accurate and meaningful textual descriptions based solely on visual inputs. This process includes fine-tuning the model with a specific dataset, optimizing its performance, and ensuring it can handle the task of converting visual information into descriptive text effectively.



Here's what we'll be covering:

[The model](#)

[Our dataset](#)

[Complex reasoning](#)

[Phi-3 Vision architecture](#)

[Preparing our dataset](#)



By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. 

[Cookies Settings](#)

Accept All

Reject All

The model

The [Phi-3-Vision-128K-Instruct](#), a lightweight, state-of-the-art multimodal model, is at the core of this project. Part of the Phi-3 model family, it supports a context length of up to 128,000 tokens. The model was trained on a diverse dataset that includes synthetic data and carefully filtered publicly available websites, emphasizing high-quality, reasoning-intensive content. The training process included supervised fine-tuning and direct preference optimization to ensure precise adherence to instructions, as well as robust safety measures.

Our dataset

The dataset used is the `DBQ/Burberry.Product.prices.United.States` dataset ([available on Huggingface](#)). It includes images of Burberry products along with metadata on the products category, price, and title with a total of 3,040 rows, each representing a unique product. This dataset lets us test the model's ability to understand and interpret visual data, generating descriptive text that capture intricate visual details and brand-specific characteristics.

Complex reasoning

One interesting aspect of this task is that the model needs to reason about prices and naming given only the image. This requires the model to not only recognize visual features but also understand their implications in terms of product value and branding. By synthesizing accurate textual descriptions from images, the project highlights the potential of integrating visual data to enhance the performance and versatility of models in real-world applications.

Phi-3 Vision architecture

The model architecture is a multimodal version of a Phi-3. It processes both text and image data, integrating these inputs into a unified sequence for comprehensive understanding and generation tasks.

The model uses separate embedding layers for text and images. Text tokens are converted into dense vectors, while images are processed through a CLIP vision model to extract feature embeddings. These image embeddings are then projected to match the text embeddings' dimensions, ensuring they can be seamlessly integrated.

Integration of text and image embeddings

Special tokens within the text sequence indicate where the image embeddings should be inserted. During processing, these special tokens are replaced with the corresponding image embeddings, allowing the model to handle text and images as a single sequence.

Here is how we will format the prompt for our dataset, using the special `<|image|>` token:

```
text = f"<|user|>\n<|image_1|>What is shown in this image?<|end|><|assistant|>\nProduct:"
```

Preparing our dataset

The dataset used in this project was sourced from Hugging Face, specifically the `DBQ/Burberry.Product.prices.United.States` dataset. To facilitate model training, the dataset was first loaded and converted into a Pandas DataFrame for easier manipulation.

Here is a script that will download the dataset and save the text to a CSV, and the images to a location on your local system:

```
import os
import pandas as pd
from datasets import load_dataset
import requests
from PIL import Image
from io import BytesIO

# Function to download an image from a URL and save it locally
def download_image(image_url, save_path):
    try:
        response = requests.get(image_url)
        response.raise_for_status() # Check if the request was successful
        image = Image.open(BytesIO(response.content))
        image.save(save_path)
        return True
    except Exception as e:
        print(f"Failed to download {image_url}: {e}")
        return False

# Download the dataset from Hugging Face
dataset = load_dataset('DBQ/Burberry.Product.prices.United.States')

# Convert the Hugging Face dataset to a Pandas DataFrame
df = dataset['train'].to_pandas()

# Create directories to save the dataset and images
dataset_dir = './data/burberry_dataset'
```

```

images_dir = os.path.join(dataset_dir, 'images')
os.makedirs(images_dir, exist_ok=True)

# Filter out rows where image download fails
filtered_rows = []
for idx, row in df.iterrows():
    image_url = row['imageurl']
    image_name = f"{row['product_code']}.jpg"
    image_path = os.path.join(images_dir, image_name)
    if download_image(image_url, image_path):
        row['local_image_path'] = image_path
        filtered_rows.append(row)

# Create a new DataFrame with the filtered rows
filtered_df = pd.DataFrame(filtered_rows)

# Save the updated dataset to disk
dataset_path = os.path.join(dataset_dir, 'burberry_dataset.csv')
filtered_df.to_csv(dataset_path, index=False)

print(f"Dataset and images saved to {dataset_dir}")

```

A crucial step in preparing the dataset involves downloading and storing the images locally.

We achieved this through a custom function, `download_image`, which fetched images from their URLs and saved them using the product codes as filenames. This method ensures that each image has a consistent and identifiable name, crucial for linking the images with the corresponding product data.

We created a CSV dataset by filtering out rows where image downloads failed to ensure dataset integrity. The DataFrame was updated with local paths of successfully downloaded images, directly linking product data with image files. This DataFrame was saved as a CSV, ready for the training process.

Training script

Now we're ready to train our Phi-3 Vision model!

The script begins by initializing essential components, including the dataset, tokenizer, and model. The dataset—split into training and validation sets—ensures effective evaluation of the model's performance during training. Additionally, we save the best validation model locally, and upload it to W&B at the end of the training run.

I always like when tutorials provide the full training script as opposed to smaller chunks,

as I find it's easier to understand the full flow of the code. Here is my training script I used to train Phi-3 Vision:

```
import os
import torch
from torch.utils.data import Dataset, DataLoader, random_split
from transformers import AutoModelForCausalLM, AutoProcessor
from torchvision import transforms
from PIL import Image
import torch.optim as optim
import pandas as pd
import random
import wandb
import torch.nn.functional as F
import numpy as np
from torchvision.transforms.functional import resize, to_pil_image

torch.manual_seed(3)

# Initialize Weights & Biases
run = wandb.init(project="burberry-product-phi3", entity="byyoung3")

# Custom Dataset for Burberry Product Prices and Images
class BurberryProductDataset(Dataset):
    def __init__(self, dataframe, tokenizer, max_length, image_size):
        self.dataframe = dataframe
        self.tokenizer = tokenizer
        self.tokenizer.padding_side = 'left'
        self.max_length = max_length

    def __len__(self):
        return len(self.dataframe)

    def __getitem__(self, idx):
        row = self.dataframe.iloc[idx]
        text = f"<|user|>\n<|image_1|>What is shown in this image?<|end|><|assistant|>\n"
        image_path = row['local_image_path']

        # Tokenize text
        encodings = self.tokenizer(text, truncation=True, padding='max_length', max_leng

    try:
```

```

        # Load and transform image
        image = Image.open(image_path).convert("RGB")
        image = self.image_transform_function(image)
    except (FileNotFoundError, IOError):
        # Skip the sample if the image is not found
        return None

    encodings['pixel_values'] = image
    encodings['price'] = row['full_price']

    return {key: torch.tensor(val) for key, val in encodings.items()}

def image_transform_function(self, image):
    image = np.array(image)
    return image

```

```

# Load dataset from disk
dataset_path = './data/burberry_dataset/burberry_dataset.csv'
df = pd.read_csv(dataset_path)

# Initialize processor and tokenizer
model_id = "microsoft/Phi-3-vision-128k-instruct"
processor = AutoProcessor.from_pretrained(model_id, trust_remote_code=True)
tokenizer = processor.tokenizer

# Split dataset into training and validation sets
train_size = int(0.9 * len(df))
val_size = len(df) - train_size
train_indices, val_indices = random_split(range(len(df)), [train_size, val_size])
train_indices = train_indices.indices
val_indices = val_indices.indices
train_df = df.iloc[train_indices]
val_df = df.iloc[val_indices]

# Create dataset and dataloader
train_dataset = BurberryProductDataset(train_df, tokenizer, max_length=512, image_size=128)
val_dataset = BurberryProductDataset(val_df, tokenizer, max_length=512, image_size=128)
train_loader = DataLoader(train_dataset, batch_size=1, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=1, shuffle=False)

# Initialize model
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="cuda", trust_remote_c

```

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Optimizer
optimizer = optim.AdamW(model.parameters(), lr=5e-5)
# Training loop
num_epochs = 1
eval_interval = 150 # Evaluate every 'eval_interval' steps
loss_scaling_factor = 1000.0 # Variable to scale the loss by a certain amount
save_dir = './saved_models'
step = 0
accumulation_steps = 64 # Accumulate gradients over this many steps

if not os.path.exists(save_dir):
    os.makedirs(save_dir)

best_val_loss = float('inf')
best_model_path = None

# Select 10 images from the validation set for logging
num_log_samples = 10
log_indices = random.sample(range(len(val_dataset)), num_log_samples)

def extract_price_from_predictions(predictions, tokenizer):
    # Assuming the price is at the end of the text and separated by a space
    predicted_text = tokenizer.decode(predictions[0], skip_special_tokens=True)
    try:
        predicted_price = float(predicted_text.split()[-1].replace(',', ''))
    except ValueError:
        predicted_price = 0.0
    return predicted_price

def evaluate(model, val_loader, device, tokenizer, step, log_indices, max_samples=None,
            model.eval()
            total_loss = 0
            total_price_error = 0
            log_images = []
            log_gt_texts = []
            log_pred_texts = []

```

```

table = wandb.Table(columns=["Image", "Ground Truth Text", "Predicted Text"])

with torch.no_grad():
    for i, batch in enumerate(val_loader):
        if max_samples and i >= max_samples:
            break

        if batch is None: # Skip if the batch is None
            continue

        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        pixel_values = batch['pixel_values'].to(device)
        labels = input_ids.clone().detach()
        actual_price = batch['price'].item()

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            pixel_values=pixel_values,
            labels=labels
        )
        loss = outputs.loss
        total_loss += loss.item()

        # Calculate price error
        predictions = torch.argmax(outputs.logits, dim=-1)
        predicted_price = extract_price_from_predictions(predictions, tokenizer)
        price_error = abs(predicted_price - actual_price)
        total_price_error += price_error

        # Log images, ground truth texts, and predicted texts
        if i in log_indices:
            log_images.append(pixel_values.cpu().squeeze().numpy())
            log_gt_texts.append(tokenizer.decode(labels[0], skip_special_tokens=True))
            log_pred_texts.append(tokenizer.decode(predictions[0], skip_special_tokens=True))

            # Convert image to PIL format
            pil_img = to_pil_image(resize(torch.from_numpy(log_images[-1]).permute(2, 0, 1)),
                                        max_size=max_size))

            # Add data to the table
            table.add_data(wandb.Image(pil_img), log_gt_texts[-1], log_pred_texts[-1])

            # Log the table incrementally

```



```
wandb.log({"Evaluation Results step {}".format(step): table, "Step": step})
```

```
avg_loss = total_loss / (i + 1) # i+1 to account for the loop index
```

```
avg_price_error = total_price_error / (i + 1)
```

```
model.train()
```

```
return avg_loss, avg_price_error
```

```
model.train()
```

```
for epoch in range(num_epochs): # Number of epochs
```

```
    total_train_loss = 0
```

```
    total_train_price_error = 0
```

```
    batch_count = 0
```

```
    for batch in train_loader:
```

```
        step += 1
```

```
        if batch is None: # Skip if the batch is None
```

```
            continue
```

```
        input_ids = batch['input_ids'].to(device)
```

```
        attention_mask = batch['attention_mask'].to(device)
```

```
        pixel_values = batch['pixel_values'].to(device)
```

```
        labels = input_ids.clone().detach()
```

```
        actual_price = batch['price'].float().to(device)
```

```
        outputs = model(
```

```
            input_ids=input_ids,
```

```
            attention_mask=attention_mask,
```

```
            pixel_values=pixel_values,
```

```
            labels=labels
```

```
        )
```

```
        loss = outputs.loss
```

```
        total_loss = loss
```

```
        predictions = torch.argmax(outputs.logits, dim=-1)
```

```
        predicted_price = extract_price_from_predictions(predictions, tokenizer)
```

```
    total_loss.backward()
```

```
    if (step % accumulation_steps) == 0:
```

```
        for param in model.parameters():
```

```
            if param.grad is not None:
```

```

        param.grad /= accumulation_steps
    optimizer.step()
    optimizer.zero_grad()

    total_train_loss += total_loss.item()
    total_train_price_error += abs(predicted_price - actual_price.item())
    batch_count += 1

    # Log batch loss to wandb
    wandb.log({"Batch Loss": total_loss.item(), "Step": step})

    print(f"Epoch: {epoch}, Step: {step}, Batch Loss: {total_loss.item()}")

    if step % eval_interval == 0:
        val_loss, val_price_error = evaluate(model, val_loader, device, tokenizer=tokenizer)
        wandb.log({
            "Validation Loss": val_loss,
            "Validation Price Error (Average)": val_price_error,
            "Step": step
        })
        print(f"Step: {step}, Validation Loss: {val_loss}, Validation Price Error (Average): {val_price_error}")

        # Save the best model
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_model_path = os.path.join(save_dir, f"best_model")
            model.save_pretrained(best_model_path, safe_serialization=False)
            tokenizer.save_pretrained(best_model_path)

    avg_train_loss = total_train_loss / batch_count
    avg_train_price_error = total_train_price_error / batch_count
    wandb.log({
        "Epoch": epoch,
        "Average Training Loss": avg_train_loss,
        "Average Training Price Error": avg_train_price_error
    })

    print(f"Epoch: {epoch}, Average Training Loss: {avg_train_loss}, Average Training Price Error: {avg_train_price_error}")

    if best_model_path:
        run.log_model(
            path=best_model_path,
            name="phi3-v-burberry",

```

```
        aliases=["best"],  
    )  
  
wandb.finish()
```

Gradient accumulation

To optimize the training process, the script incorporates several key techniques. Gradient accumulation is used to handle large batches efficiently, crucial given the computational demands of multimodal models. This technique allows the model to accumulate gradients over multiple steps before performing a weight update, effectively simulating a larger batch size and stabilizing the training process.

Here's how we do gradient accumulation:

```
if (step % accumulation_steps) == 0:  
    for param in model.parameters():  
        if param.grad is not None:  
            param.grad /= accumulation_steps  
    optimizer.step()  
    optimizer.zero_grad()
```

Evaluation

During training, we evaluate the model's performance at regular intervals, and save the best-performing model (based on validation loss). This approach ensures that the final model retains the most effective parameters learned during training.

Special emphasis is placed on logging the price prediction errors to monitor the model's performance in predicting accurate prices. This detailed tracking helps in understanding how well the model is learning to predict prices based solely on images, a complex task requiring deep visual understanding.

The training loop iterates through the dataset, processing batches of text and image data. For each batch, the model's predictions are compared with the actual data, and the loss is computed. The total loss is then backpropagated, and gradients are accumulated. After a set number of steps, the accumulated gradients are used to update the model's weights.

Evaluation is a critical component of the training script. At specified intervals, the model is evaluated on the validation set, which gives us the ability to see how the model is generalizing to unseen data. The evaluation function calculates the average validation loss and price prediction error, logging these metrics to W&B.

The following code allows us to log evaluation metrics to W&B:

```
if step % eval_interval == 0:
    val_loss, val_price_error = evaluate(model, val_loader, device, tokenizer=tokenizer)
    wandb.log({
        "Validation Loss": val_loss,
        "Validation Price Error (Average)": val_price_error,
        "Step": step
    })
```

We track “price error” to evaluate the model's accuracy in predicting product prices. By measuring the difference between predicted and actual prices, we can quantitatively identify how well the model performs at predicting prices, going beyond a traditional ‘next token’ loss, which is a bit vague for a task like price prediction. This ensures the model is reliable for real-world pricing applications.

We select a group of ten samples from the validation set track them across multiple validation checkpoints during the training process. These samples are logged to a table, including images, ground truth texts, and predicted texts, providing a comprehensive view of the model's performance over time. These logs, along with the quantitative metrics, offer a thorough evaluation of the model's ability to generate accurate and meaningful outputs.

Then, we create a table containing the image, ground truth labels, and the model's predicted text at that particular stage in the training run, and finally log this table to W&B with the following code:

```
def evaluate(model, val_loader, device, tokenizer, step, log_indices, max_samples=None):
    model.eval()
    total_loss = 0
    total_price_error = 0
    log_images = []
    log_gt_texts = []
    log_pred_texts = []
    table = wandb.Table(columns=["Image", "Ground Truth Text", "Predicted Text"]) # initialize table

    with torch.no_grad():
        for i, batch in enumerate(val_loader):
            if max_samples and i >= max_samples:
                break
```

```

if batch is None: # Skip if the batch is None
    continue

input_ids = batch['input_ids'].to(device)
attention_mask = batch['attention_mask'].to(device)
pixel_values = batch['pixel_values'].to(device)
labels = input_ids.clone().detach()
actual_price = batch['price'].item()

outputs = model(
    input_ids=input_ids,
    attention_mask=attention_mask,
    pixel_values=pixel_values,
    labels=labels
)
loss = outputs.loss
total_loss += loss.item()

# Calculate price error
predictions = torch.argmax(outputs.logits, dim=-1)
predicted_price = extract_price_from_predictions(predictions, tokenizer)
price_error = abs(predicted_price - actual_price)
total_price_error += price_error

# Log images, ground truth texts, and predicted texts
if i in log_indices:
    log_images.append(pixel_values.cpu().squeeze().numpy())
    log_gt_texts.append(tokenizer.decode(labels[0], skip_special_tokens=True))
    log_pred_texts.append(tokenizer.decode(predictions[0], skip_special_tokens=True))

    # Convert image to PIL format
    pil_img = to_pil_image(resize(torch.from_numpy(log_images[-1]).permute(2, 0, 1)))

    # Add data to the table
    table.add_data(wandb.Image(pil_img), log_gt_texts[-1], log_pred_texts[-1])

    # Log the table incrementally

wandb.log({"Evaluation Results step {}".format(step): table, "Step": step})

avg_loss = total_loss / (i + 1) # i+1 to account for the loop index
avg_price_error = total_price_error / (i + 1)
model.train()

```

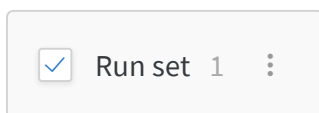
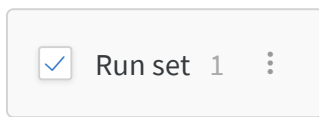
```
return avg_loss, avg_price_error
```

Model logging

The best model—determined by the lowest validation loss—is saved locally. After the training is complete, this best model is then logged to W&B using `run.log_model()`. Logging our model is a great way to ensure we have a central location for all of our models.

By the end of the training process, the model demonstrates a huge improvement in its capability to generate product metadata given images, effectively predicting product prices along with other product information like title and category.

Here are the logs for my training run:



Utilizing W&B Registry

Once we have saved our model to W&B, we can add it to our W&B Registry. The Registry in W&B is a centralized repository that allows us to manage and version our machine learning models. It helps track model lineage, compare different versions, and deploy the best-performing models seamlessly.

First, we navigate to the artifacts pane shown in our run page. You should see a row that

looks like this:

model	phi3-v-burberry:v0	17
-------	--------------------	----

Click this row, and you will be redirected to another page that looks like this:

Artifacts

Model Registry →

Find matching artifacts

model

- phi3-v-burberry
 - v0 best latest
- wandb-history
 - run-fqrheyvi-history
 - run-3uxj93gh-history
- run_table

phi3-v-burberry

Version 0

Version Metadata Usage Files Lineage

Version overview

Full Name

byyoung3/burberry-product-phi3/phi3-v-burberry:v0

Created At

May 24th, 2024 16:18:42

Aliases

@ best × @ latest @ v0 +

Num Consumers

0

Digest

a6fa0fc319916658e17c312245eef927

Num Files

8

Created By

fanciful-sky-2

Size

8.3GB

Description

What changed in this version?

TTL Remaining

Inactive

Link to registry

At the top right, you will see a button called “Link to registry” which will allow us to add the model to our registry. After clicking this button, you will be presented with the option to add it to an existing model, or create a new model. Assuming you have not created the model, simply click register a new model.

Link to registry

Step 1 of 2 - Model details

Select a collection

Select an artifact

Models

- phi3-v-burberry
 - 1 version

+ Register new model

After registering the model, navigate to the Registry page, and you will see your model:

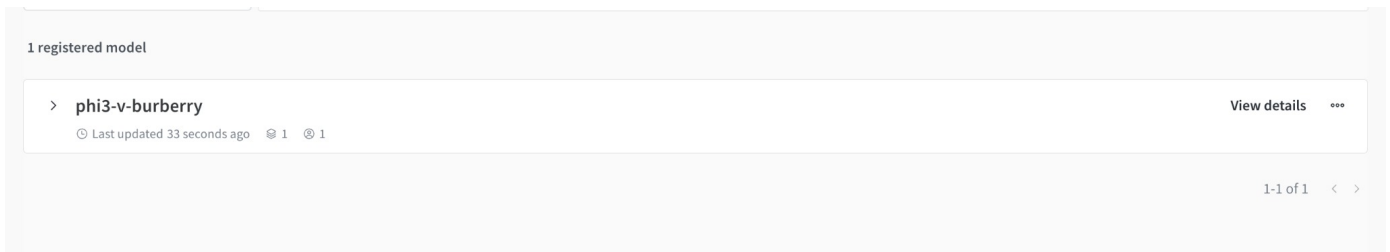
Model registry

Documentation

+ New registered model

byyoung3

Search model name and tags



Running inference with Phi-3 Vision

Now that we have successfully added the model to our Registry, we can now access our saved model programmatically, and run inference.

Here's the script that will allow us to accomplish this:

```
import weave
import os
import torch
from transformers import AutoModelForCausalLM, AutoProcessor
from PIL import Image
import requests
from io import BytesIO
import base64
from pathlib import Path
import wandb

# Initialize Weights & Biases run
run = wandb.init(project='burberry-product-price-prediction')
artifact = run.use_artifact('byyoung3/model-registry/phi3-v-burberry:v0', type='model')
artifact_dir = artifact.download()
print(f"Artifact downloaded to: {artifact_dir}")

model_id = "microsoft/Phi-3-vision-128k-instruct"

try:
    model = AutoModelForCausalLM.from_pretrained(
        artifact_dir,
        torch_dtype=torch.float16,
        attn_implementation="flash_attention_2",
        trust_remote_code=True
    )
    processor = AutoProcessor.from_pretrained(model_id, trust_remote_code=True)
except Exception as e:
    print(f"Error loading model or processor: {e}")
    raise

# Ensure the model is on the correct device
```

```

device = 'cuda'
model.to(device)
# Function to run inference on a single image
@weave.op
def run_inference(image_url: str) -> dict:
    try:
        prompt = "<|user|>\n<|image_1|>What is shown in this image?<|end|><|assistant|>\n"
        # Load image
        image = Image.open(requests.get(image_url, stream=True).raw)
        inputs = processor(prompt, [image], return_tensors="pt").to(device)
        generation_args = {
            "max_new_tokens": 500,
            "temperature": 0.0,
            "do_sample": False,
        }

        generate_ids = model.generate(**inputs, eos_token_id=processor.tokenizer.eos_token_id)

        # Remove input tokens
        generate_ids = generate_ids[:, inputs['input_ids'].shape[1]:]
        response_text = processor.batch_decode(generate_ids, skip_special_tokens=True, clean_up_tokenization_spaces=False)

        return {
            "predicted_text": response_text,
            "image": image
        }

    except Exception as e:
        print(f"Error during inference: {e}")
        raise

# Initialize Weave project
weave.init('burberry-product-price-prediction')

# Example usage
image_url = "https://assets.burberry.com/is/image/BurberryLtd/1C09D316-7A71-472C-8877-91"
try:
    result = run_inference(image_url)
    print("Predicted Text:", result['predicted_text'])
except Exception as e:
    print(f"Error running inference: {e}")

```

The provided script initializes a Weights & Biases run and downloads a model artifact.

The script includes a function, `run_inference`, which takes an image URL, fetches the image, processes it, and runs inference using the loaded model. The function constructs a prompt, processes the image and prompt into input tensors, and generates a textual response from the model. An example usage demonstrates running inference on a Burberry product image, printing the model's predicted text.

Additionally, we have integrated W&B Weave into our inference script, which logs outputs of our model! Weave is a lightweight toolkit by Weights & Biases for tracking and evaluating language model (LLM) applications. By decorating Python functions with `@weave.op`, Weave helps log and debug model inputs and outputs, build evaluations, and organize information from experimentation to production. In our example, we use Weave to load a model from Weights & Biases, run inference on images, and use Weave to log the images and model responses.

Here's what it looks like in Weave after running our inference script:

run_inference 46b6

Call

Code

Feedback

Summary

Use

Inputs

Path

Value

image_url

[https://assets.burberry.com/is/image/BurberryLtd/1C09D316-7A71-472C-8877-91CEFBDB268A?\\$BBY_V3_SL_1\\$&wid=1501&hei=1500](https://assets.burberry.com/is/image/BurberryLtd/1C09D316-7A71-472C-8877-91CEFBDB268A?$BBY_V3_SL_1$&wid=1501&hei=1500)

Text

Output

Path

Value

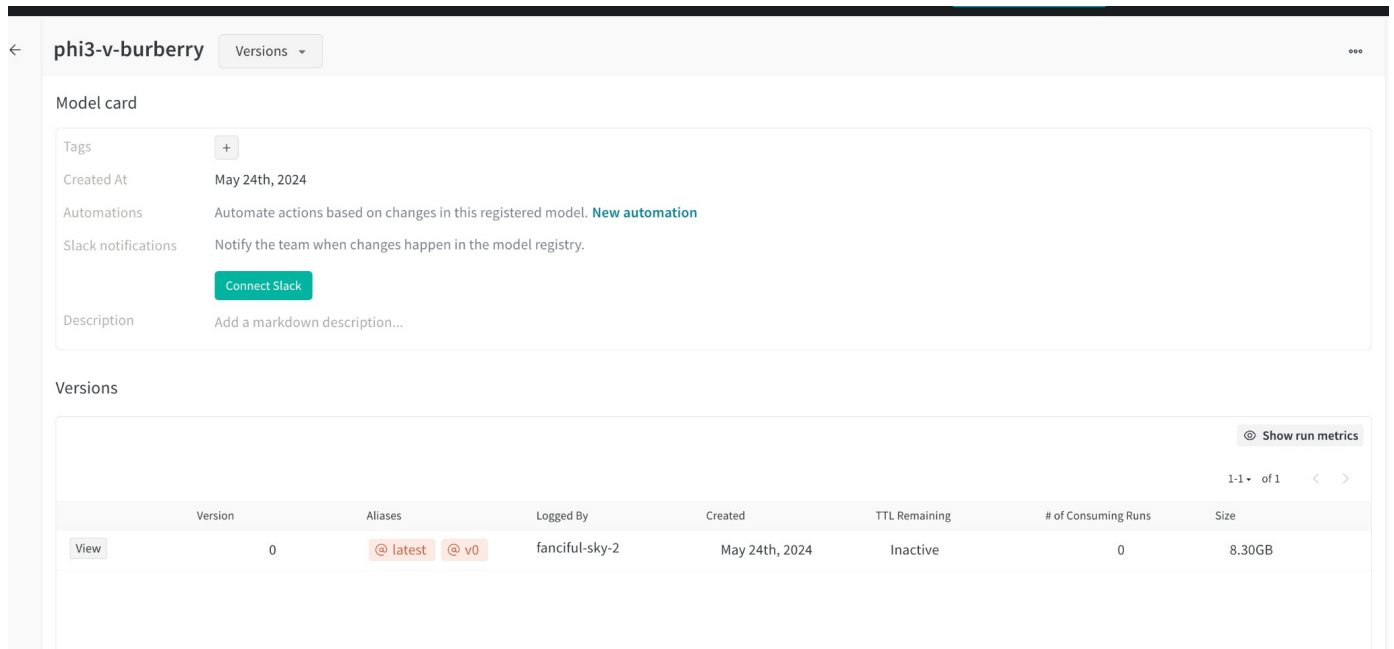
predicted_text

Product: Check Hooded Jacket, Category: HOODIES & SWEATSHIRTS, Full Price: 1790.0

image



Slack integration

Now, I want to show off a really cool feature of W&B. Hypothetically, let's say we have another team responsible for model evaluation, and we would like to notify the eval team every time we upload a new model to our W&B Registry, so that the team can begin evaluating the model. Additionally, let's assume the team uses Slack. W&B provides an awesome integration with Slack and Registry, so we can automate the process of letting our evaluation team know that a new model is ready to be evaluated! Simply click the registered model in the Registry page, and you will be presented with the following: 

← phi3-v-burberry Versions ▾

Model card

Tags

+

Created At

May 24th, 2024

Automations

Automate actions based on changes in this registered model. [New automation](#)

Slack notifications

Notify the team when changes happen in the model registry.

Connect Slack

Description

Add a markdown description...

Versions

☼ Show run metrics

1-1 of 1 < >

Version	Aliases	Logged By	Created	TTL Remaining	# of Consuming Runs	Size
View 0	@ latest @ v0	fanciful-sky-2	May 24th, 2024	Inactive	0	8.30GB

By clicking the “Connect Slack” button, you will be able to connect the registry to a slack channel of your choosing! When new models get added to your registry, you will now get Slack notifications:



Weights & Biases APP 6:35 PM

New model version linked to the model registry

Artifact version phi3-v-burberry:0 linked to phi3-v-burberry

Conclusion

This project demonstrates the capability of the Phi-3-Vision-128K-Instruct model in processing and synthesizing text from image data. The model's ability to generate accurate and meaningful textual descriptions from visual inputs is a testament to its sophisticated design and training.

By working with a dataset that includes detailed information on Burberry products—encompassing categories, images, prices, and titles—the model has shown it can understand and interpret visual data, even making inferences about prices and product naming from images alone. This task is particularly intriguing as it requires the model to not only recognize and process visual features but also to understand their implications in terms of product value and branding.

Moreover, the project highlights the seamless integration with Weights & Biases for model artifact management and tracking model predictions during inference. By saving the best model to the W&B Registry, it ensures easy access and version control. The integration with Slack further enhances the workflow, allowing for automated notifications whenever a new model is uploaded to the registry. This automated communication ensures that updates are promptly shared, streamlining the process. Overall, I hope you enjoyed this tutorial, and if you are interested in the source code, you can find it [here](#)!

Related Articles



Building a real-time answer engine with Llama 3.1 405B and W&B Weave

Infusing llama 3.1 405B with internet search capabilities!!

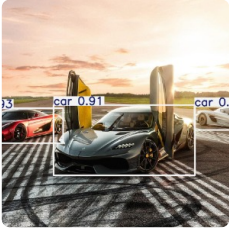


Grokking: Improved generalization through overfitting

One of the most mysterious phenomena in deep learning; Grokking is the



tendency of neural networks to improve generalization by sustained ...



YOLOv9 object detection tutorial

How to use one of the worlds fastest and most accurate object detectors to run inference, display on your webcam using OpenCV and tracking your ...



Fine-Tuning Llama-3 with LoRA: TorchTune vs HuggingFace

A battle between the HuggingFace and TorchTune!!!

Add a comment



alli prashanth • 6 months ago

how to run the script locally using CPU? what are the minimum requirements for RAM and CPU

B 1 reply



Denis Lemarchand • 7 months ago

Very cool tutorial. Thank you ! I've adapted it for a personal project. I wonder which GPU was used for training? I use google colab for my experiments, I had to quantize the model in 4 bits. The results are not up to my expectations, so I'm wondering if it's the effect of the quantization.

B 1 reply

Tags: Articles, LLM, GenAI, Text Generation, Intermediate, Computer Vision

New course
LLM Apps: Evaluation

ENROLL NOW →



Weights
& Biases



Google

Made with Weights & Biases. [Sign up](#) or [log in](#) to create reports like this one.

Never lose track of
another ML project.
Try Weights & Biases today.

SIGN UP

TRY W&B NOW



Weights & Biases

Get weekly updates with the latest ML news.

Subscribe

PRODUCTS

Dashboard | Sweeps | Artifacts | Reports | Tables

QUICKSTART

Documentation

RESOURCES

Courses | Forum | Tutorials | Benchmarks

W&B

